

Author Response for Submission “PushAround: Collaborative Path Clearing via Physics-Informed Hybrid Search”

Author Names Omitted for Anonymous Review.

To Associate Editor

Q: *The reviewers acknowledge the contribution of the paper, however, they have some concerns that need to be addressed.*

A: We sincerely thank the Associate Editor for the careful handling of our submission and for summarizing the reviewers’ constructive comments. In response, we have substantially revised the manuscript to clarify the methodological details, implementation choices, and experimental limitations identified by the reviewers. We also added targeted evaluations of planner–execution mismatch, obstacle-density scaling, robot-team scaling, and WCCG connectivity. The principal revisions are summarized below.

Table 1: New and revised experiments at a glance; page numbers refer to the revised manuscript.

Study	Setting	PushAround (ours)	Strongest baseline	Location
Main comparison	8×5 m, $M=10$, $N=2$; 40 trials	97.5 %	SL-Push (sim) 75.0 %	Table II, p. 6
Ablations	Scenario 1, 40 trials	82.5–97.5 %	—	Table II, p. 6
Obstacle density	8×8 m, $M=15$ –45, $N=2$; 10 seeds each	100 % for all M	0/10 at $M=45$	Table III, Fig. 10; p. 7
Physics mismatch	8×8 m, $M=30$, $N=2$; $\pm 20\%$ and $\pm 40\%$; 10 seeds per condition (ours), 5 for the baseline supplements at the $\pm 40\%$ extremes	97.5 % at $\pm 40\%$	DFS-WCCG 57.5 %; SL-Push (sim) 32.5 %	Table III, p. 7
Robot-team scaling	8×8 m, $M=30$, $N=2$ –4; 10 seeds each	100 % (30/30)	DFS-WCCG 90–100 %; SL-Push (sim) 80 %	Table III, p. 7
WCCG validation	300 random scenes, incl. non-convex	300/300, 0 FP/FN	—	Table I, p. 6

The principal revisions are:

- **Physics-model mismatch (new experiment).** The planner uses the nominal model while only the execution simulator is perturbed, one parameter family at a time, by factors covering $\pm 20\%$ and $\pm 40\%$. PushAround succeeds in **156/160** perturbed trials (**97.5%**), or **166/170** (**97.6%**)

including the nominal condition, whereas under the same perturbations SL-Push (sim) reaches **13/40** and DFS-WCCG **23/40** (Sec. IV-A6, Table III, Fig. 1). This study is new: the first submission validated feasibility only under nominal physics, so the sensitivity of the method to an incorrect planning model had not been characterized before.

- **Density-scaling statistics (revised experiment).** The density sweep now uses **10 independent seed sequences** per density instead of five. PushAround maintains **100%** success for every $M \in [15, 45]$ with mean planning time rising from 2.5 to 53.4 s, while the comparison methods fall to 0/10 at $M=45$; dispersion and Wilson intervals are now reported (Sec. IV-A6, Table III, Fig. 7). The no-recursion ablation is reported alongside, since it isolates how much of the dense-clutter success is attributable to recursive pushing.
- **Robot-team scaling (new experiment and mechanism).** Two mode-generation refinements were added to the shared generator, which all policies use: bounded backtracking contact assignment and a temporary per-task robot release. At $N=4$ the observed success rates change from 70% to **100%** for PushAround, from 0% to 80% for SL-Push (sim) and from 80% to 90% for DFS-WCCG. PushAround succeeds in all 30 team-size trials at 29–30 s of planning and 97–110 s of execution, versus 102–143 s and 176–178 s for DFS-WCCG (Sec. III-C2, Sec. IV-A6, Table III, Fig. 9, and Fig. 4).
- **WCCG connectivity validation (new experiment).** Sec. III-A2 derives the criterion from $W/2$ configuration-space inflation with explicit non-convex decomposition. On 300 scenes WCCG agrees with the continuous reference in **300/300** cases with **0 FP/FN**, while the continuous and rasterized references agree in 296/300 cases; WCCG averages **1.07 ms** per query versus 1.36 ms (CC) and 243.1 ms (FR) (Table I and Fig. 6).
- **Formulation, guarantees and interfaces.** Eq. (5) is now presented as the schedule cost guiding the search; the paper states explicitly that PIHS is a feasibility-oriented heuristic search with finite sampling, bounded lookahead, a non-admissible priority, and first-feasible termination, and makes no global optimality or completeness claim. The optimized contact forces are declared feasibility variables (not hardware setpoints), and coupled object–object motion is explained as an outcome of the joint rigid-body rollout in EvalSim (Sec. II-C, II-D, III-C2, III-C3, III-D, III-E1).
- **Comparison protocol and budgets.** Sec. IV-A1 now states the shared footprints, obstacle configurations, collision/reachability checks, success criterion and simulator, and the shared per-trial budget (**800 s** timeout, **20,000** control steps, **8** parallel workers), with unsuccessful trials assigned the 800 s cap in planning-time statistics. The previous 400 s statement is corrected.

The detailed responses below provide the manuscript locations and revised text for each reviewer comment. The supplementary evaluation package has also been updated with the per-trial records and configuration files backing the reported statistics.

To Reviewer 3

Q: *This paper presents PushAround, a physics-informed hybrid search framework for collaborative path clearing in environments containing multiple movable obstacles. The problem is formulated within NAMO, but specifically considers a team of mobile robots that actively rearrange obstacles to create a corridor with sufficient clearance for a larger external vehicle.*

A: We sincerely thank the reviewer for the positive assessment and, in particular, for recognizing the end-to-end integration, recursive pushing, dense-clutter scalability, and hardware validation. The revision addresses the two stated weaknesses and the four follow-up questions without changing the central planning framework. The detailed responses below explain these revisions point by point and provide the exact modified text highlighted in blue. The supplementary evaluation package, including the updated simulation and hardware materials, has also been updated.

Q: *Strengths: - Coherent end-to-end framework; The integration of WCCG, gap ranking, pushing-mode generation, and physics validation is well designed. The method jointly considers which obstacle to move and how to push it, while validating candidate actions through simulation.*

- Recursive pushing; The planner can first clear movable blockers in the swept region and then return to the original gap-opening task. Good feature!

- Scalability: The method is evaluated with up to 45 movable obstacles, which is a useful demonstration of scalability in dense clutter. The reported success remains at 100% at 45 obstacles, although the experiment uses five seeds per density.

- Strong simulation results and real-world validation: The proposed method substantially outperforms the reported baselines in simulation and is also demonstrated with two real robots.

A: We thank the reviewer for these observations. The revision keeps the end-to-end structure unchanged and strengthens the three aspects that the revision touches: the recursive-pushing mechanism is now illustrated by its own figure (Fig. 4, Page 4) and remains a distinguishing feature of the planner; the dense-clutter evaluation now uses ten independent seed sequences per density instead of five (Sec. IV-A6, Page 7), which addresses the caveat attached to the scalability result; and the two-robot hardware demonstration, including the WCCG overlays of the executed runs, is retained in Fig. 11 (Page 8) with the execution monitor now specified explicitly.

Q: *Physics-model robustness is insufficiently characterized: The method relies heavily on simulation-based feasibility validation, but sensitivity to uncertainty in mass, friction, force limits, contact dynamics etc. is not systematically evaluated.*

A: We agree that this was the most substantive gap, because the feasibility validation is simulation based. The original hardware experiments demonstrated closed-loop recovery but did not *isolate* sensitivity to individual model parameters. The revision therefore adds a dedicated planner–execution mismatch study designed to separate model mismatch from model re-identification:

- The planner always uses the nominal model; *only* the execution simulator is perturbed. The experiment therefore measures the consequence of an incorrect planning model rather than the benefit of re-identifying it.
- Obstacle mass, lateral friction, spinning friction and the robot force limit are scaled separately by $\{0.6, 0.8, 1.2, 1.4\}$, i.e. $\pm 20\%$ and $\pm 40\%$, using the same scene seeds as the nominal condition.

Across 17 conditions ($M=30$, $N=2$, 10 seeds each, 170 trials), PushAround succeeds in **166/170 (97.6%)**. The perturbed subset contains **156/160 successes (97.5%)**: the aggregate rate is 78/80 (97.5%) at $\pm 20\%$ and 78/80 (97.5%) at $\pm 40\%$, compared with 10/10 nominal. The four failures comprise one mid-execution planning failure (lateral friction $\times 0.6$) and three trials that exceeded the per-trial budget (spin friction $\times 1.2$, force limit $\times 0.6$ and $\times 1.2$). Mean replans increase from 2.3 (nominal) to 3.9 (averaged over the $\pm 40\%$ conditions; up to 8.1 under force limit $\times 0.6$), while mean execution time increases from 104.7 s to 159.6 s in the latter condition.

For context, SL-Push (sim) and DFS-WCCG were also evaluated under nominal planning and execution-only perturbations, using five seeds per condition. Under the eight $\pm 40\%$ extremes, SL-Push (sim) succeeds in **13/40 trials (32.5%)** and DFS-WCCG in **23/40 trials (57.5%)**. Their mean planning times are 31.5 s and 229.1 s, respectively, compared with 18.4 s for PushAround. Because the baseline supplement uses fewer seeds and has lower nominal success (3/5 for each baseline), these results provide end-to-end context rather than a normalized estimate of each method’s degradation from its own nominal performance.

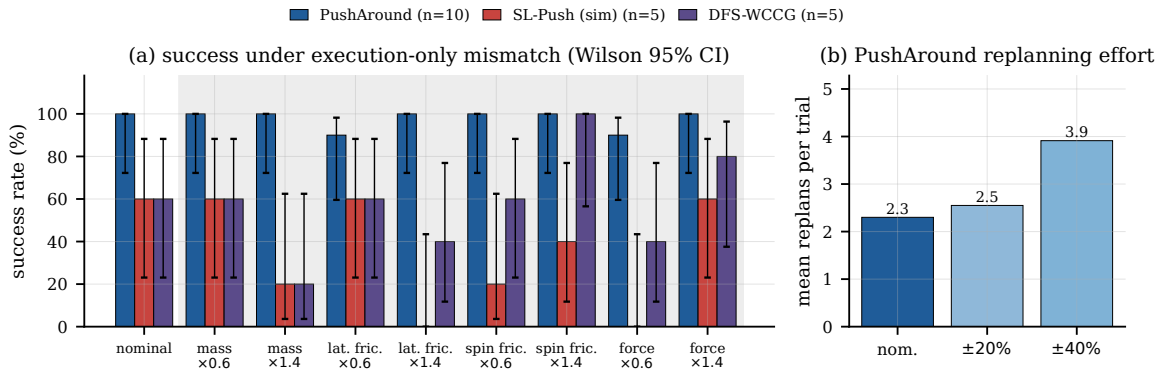


Figure 1: Planner-execution mismatch. (a) Success rate for PushAround ($n=10$ per condition) and for the two baselines ($n=5$) under execution-only perturbations; whiskers are Wilson 95% intervals. (b) Mean number of replans per trial for PushAround, showing that online replanning, rather than a retuned model, absorbs the mismatch.

Changes in the revised manuscript. Sec. I-B (Page 1) now motivates the mismatch evaluation, while Sec. IV-A6 and Table III (Page 7) define the protocol and report the perturbed and nominal aggregates:

- The revised introduction now states that model fidelity affects physics validation and that execution-model mismatch is evaluated explicitly.

Since physics validation depends on model fidelity, robustness under execution-model mismatch is also evaluated. (Sec. I-B) and Additional validation characterizes connectivity-query consistency and sensitivity to physics-model mismatch. (abstract)

- Sec. IV-A6 and Table III report 156/160 successes for the perturbed PushAround trials, distinguish the nominal condition, and give the baseline supplement results at the $\pm 40\%$ extremes.

the planner keeps the nominal model and only the execution simulator is perturbed, one parameter at a time: PushAround succeeds in 156 of 160 perturbed trials (97.5%), the four failures being one low-friction planning failure and three budget timeouts.

Table 2: Complete planner–execution mismatch grid. PushAround uses 10 seeds per condition; the two baselines 5 seeds, evaluated on the nominal condition and the eight $\pm 40\%$ extremes. PT/ET are means over successful trials, $\bar{R}$ the mean number of replans.

Family	Factor	PushAround ($n=10$)				SL-Push (sim) ($n=5$)			DFS-WCCG ($n=5$)	
		succ.	PT	ET	$\bar{R}$	succ.	PT	ET	succ.	PT
Nominal	1.0	10/10	16.8	104.7	2.3	3/5	33.6	310.9	3/5	156.2
Mass	0.6	10/10	15.9	96.2	2.3	3/5	30.9	300.4	3/5	205.6
Mass	0.8	10/10	15.3	95.0	3.5	—	—	—	—	—
Mass	1.2	10/10	16.3	108.5	2.7	—	—	—	—	—
Mass	1.4	10/10	19.8	124.1	4.4	1/5	40.2	319.3	1/5	136.5
Lateral friction	0.6	9/10	16.5	130.2	3.4	3/5	34.4	315.6	3/5	222.1
Lateral friction	0.8	10/10	14.0	100.0	2.8	—	—	—	—	—
Lateral friction	1.2	10/10	15.8	110.9	2.7	—	—	—	—	—
Lateral friction	1.4	10/10	19.8	116.5	4.0	0/5	—	—	2/5	323.2
Spin friction	0.6	10/10	14.1	91.1	2.7	1/5	28.9	251.4	3/5	179.7
Spin friction	0.8	10/10	15.4	100.9	2.0	—	—	—	—	—
Spin friction	1.2	9/10	14.9	98.9	2.2	—	—	—	—	—
Spin friction	1.4	10/10	16.5	109.2	3.2	2/5	27.3	222.6	5/5	237.0
Force limit	0.6	9/10	29.7	159.6	8.1	0/5	—	—	2/5	360.3
Force limit	0.8	10/10	16.9	121.3	3.0	—	—	—	—	—
Force limit	1.2	9/10	14.7	91.4	1.5	—	—	—	—	—
Force limit	1.4	10/10	15.2	93.3	3.2	3/5	27.6	301.9	4/5	168.1
Aggregate $\pm 20\%$		78/80	15.4	103.4	2.6	—	—	—	—	—
Aggregate $\pm 40\%$		78/80	18.4	115.0	3.9	13/40	31.5	285.2	23/40	229.1
Aggregate all		166/170	16.9	108.9	3.2	16/45	31.8	288.9	26/45	221.0

Q: *Limited evaluation of more complex scenarios: Although the method is tested with up to 45 movable obstacles, the number of independent trials is limited.*

A: We agree, and we have both increased the number of independent runs and changed how the results are reported.

- The density sweep now uses **10 independent seed sequences** per density instead of five (7 densities $\times$ 10 seeds = 70 PushAround trials). For each seed the denser instances are generated incrementally, while different seeds are generated independently, so the success-rate estimates at neighbouring densities are paired but the seeds themselves are not.
- Fig. 10 now shows the success rate and the mean planning time with one-standard-deviation whiskers; Table III lists PT and ET as mean $\pm$ s.d. Failed and pruned baseline trials are assigned the 800 s cap before averaging, and the failure composition (plan failure vs. timeout vs. timeout-pruned) is reported per density in the appendix tables.
- All headline success rates are reported with Wilson 95% intervals in Fig. 7. For PushAround at $M=45$ the interval is $[72.2, 100]\%$ from 10/10; for the baselines at the same density it is $[0, 27.8]\%$ from 0/10.

Changes in the revised manuscript. Sec. IV-A6 and the caption of Fig. 10 (Page 7) now state the seed count, nested construction within a seed, and dispersion. Table III (Page 7) reports the corresponding mean $\pm$ s.d. values:

- Sec. IV-A6 and the caption of Fig. 10 now state the seed count, the nested construction within a seed, and the dispersion:

Obstacle-density scaling (8×8 m, 10 independent seeds per density). (a) Success rate; (b) mean planning time, with unsuccessful trials capped at the 800 s budget (dotted line); (c) mean

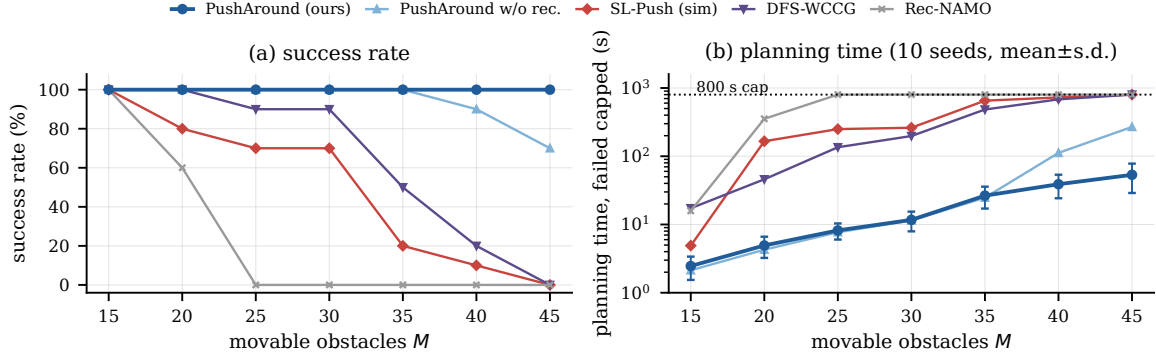


Figure 2: Obstacle-density scaling with 10 independent seeds per density. (a) Success rate; PushAround remains at 100% over the whole range. (b) Planning time with failed trials capped at the 800 s budget; whiskers denote one standard deviation over seeds. Rec-NAMO reaches the cap from $M=25$ onward, and DFS-WCCG and SL-Push (sim) reach it at $M=45$.

Table 3: Density scaling, success rate (%) and capped mean planning time (s); 7 densities $\times$ 10 seeds per policy. The full table including execution time, push count and failure composition is given in Table 4.

	$M=15$	20	25	30	35	40	45
PushAround (succ.)	100	100	100	100	100	100	100
PushAround (PT)	2.5	4.9	8.2	11.7	26.5	38.9	53.4
PushAround w/o rec. (succ.)	100	100	100	100	100	90	70
PushAround w/o rec. (PT)	2.1	4.3	7.7	11.4	24.7	112.0	268.9
SL-Push (sim) (succ.)	100	80	70	70	20	10	0
SL-Push (sim) (PT)	4.9	165.9	249.9	261.5	652.5	727.0	800.0
DFS-WCCG (succ.)	100	100	90	90	50	20	0
DFS-WCCG (PT)	17.3	45.9	135.5	197.7	483.8	681.7	800.0
Rec-NAMO (succ.)	100	60	0	0	0	0	0
Rec-NAMO (PT)	15.8	353.5	800.0	800.0	800.0	800.0	800.0

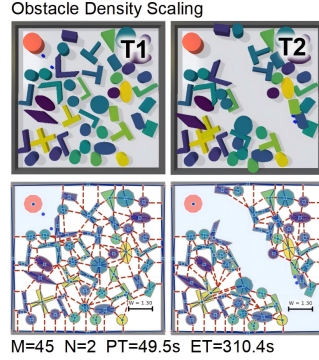


Figure 3: Representative dense-clutter scenes of the density sweep ($M=45$, two robots): snapshots during clearing above and the corresponding WCCG overlays with the start face in blue below.

execution time of the successful trials (Rec-NAMO has none beyond $M=20$). Error bars denote one standard deviation across seeds.

- Table III reports PT/ET as mean $\pm$ s.d. for the representative densities $M \in \{15, 30, 45\}$.

Table 4: Complete obstacle-density grid (10 seeds per cell). Top: success rate (%). Bottom: mean planning time with unsuccessful trials capped at 800 s. PushAround additionally reports execution time and push count.

Success (%)	$M=15$	20	25	30	35	40	45
PushAround	100	100	100	100	100	100	100
PushAround w/o rec.	100	100	100	100	100	90	70
SL-Push (sim)	100	80	70	70	20	10	0
DFS-WCCG	100	100	90	90	50	20	0
Rec-NAMO	100	60	0	0	0	0	0
Planning time (s)	$M=15$	20	25	30	35	40	45
PushAround	2.5 ± 0.9	4.9 ± 1.7	8.2 ± 2.2	11.7 ± 3.7	26.5 ± 9.3	38.9 ± 14.7	53.4 ± 24.5
PushAround w/o rec.	2.1 ± 0.8	4.3 ± 1.3	7.7 ± 2.1	11.4 ± 3.0	24.7 ± 8.2	112.0 ± 242.0	268.9 ± 366.6
SL-Push (sim)	4.9 ± 4.8	165.9 ± 334.2	249.9 ± 379.7	261.5 ± 371.9	652.5 ± 311.5	727.0 ± 230.9	800.0
DFS-WCCG	17.3 ± 9.1	45.9 ± 47.5	135.5 ± 241.1	197.7 ± 248.4	483.8 ± 339.6	681.7 ± 253.5	800.0
Rec-NAMO	15.8 ± 12.4	353.5 ± 385.5	800.0	800.0	800.0	800.0	800.0
PushAround: <i>ET</i> (s)	37.1	56.9	77.3	100.9	165.4	213.2	272.7
PushAround: pushes	3.5	5.8	7.9	10.5	16.6	20.7	26.1

Q: *The experiments primarily use two robots. What happens when the number of robots increases with the complexity of the task? How does the search and the contact/force formulation scale to larger robot teams?*

A: This comment revealed a bottleneck in the shared mode-generation module. We added two local refinements and re-ran PushAround and both baselines with the same updated generator; the resulting comparison is reported below.

For complexity, with $|\mathcal{A}|$ reachable contact candidates per robot, an exhaustive joint assignment would require $|\mathcal{A}|^N$ tuples. The greedy stage instead tests at most $N|\mathcal{A}|$ single-contact substitutions; when it fails, the bounded-backtracking search expands at most $B=2000$ partial assignments — a fixed budget that does not grow with the team size — and the subset fallback repeats that search for at most one excluded robot. Each resulting candidate is checked by a force LP with a $2N$ -dimensional decision vector. The number of contact assignments examined is bounded by $N|\mathcal{A}|+2B$, rather than growing as $|\mathcal{A}|^N$; each examined assignment requires an LP over $2N$ force variables, so team size affects both the number and the dimension of the LPs, while the implementation avoids exhaustive enumeration of joint contact tuples. Sec. III-E2 states this distinction and the budget used.

The preliminary $N=4$ runs failed in 3 of 10 seeds (2 planning failures and 1 budget timeout). The diagnosis was that, for some push tasks *no* full-team contact assignment exists — small pushable faces, or the pairwise pusher-clearance constraint rejecting every remaining candidate of one robot — and the legacy greedy replacement was additionally sensitive to robot ordering. We added two mode-generation-local refinements that leave global planning, task sampling and the search tree unchanged: (i) *bounded backtracking* contact assignment and (ii) a *temporary per-task robot release* (up to one robot is excluded for that single push and holds its pose, a penalty is charged during mode ranking so full-team modes are preferred, and the full team is re-available for every later task).

Both refinements are in the shared mode generator, so the baseline cells were re-run with the same code. Across their 40 trial logs, instrumentation records 20,890 failed greedy replacements, 10,904 bounded-backtracking assignments (of which 739 rescue a failed greedy replacement), and 6,017 per-task robot releases. These counts confirm that the added paths are exercised by the baselines as well as by PushAround:

After the refinement, all three methods solve 80–100% of the team-size trials, and the success-rate intervals at $N=3, 4$ overlap (at $N=4$: PushAround $[0.72, 1.00]$, DFS-WCCG $[0.60, 0.98]$, and SL-Push (sim) $[0.49, 0.94]$). We therefore make no monotone team-size claim. The observed separation

Table 5: Robot-team scaling at $M=30$ (nominal physics) after the shared mode-generation fix. Every cell uses 10 seeds and the same shared mode-generation implementation and evaluation protocol; PT/ET are means over successful trials. For reference, the pre-fix history is given in the notes.

Policy	N	Trials	Succ.	PT (s)	ET (s)	pushes	sims
PushAround	2	10	10/10	23.9 ± 10.9	105.0 ± 35.9	11.7	81
	3	10	10/10	30.0 ± 12.7	97.3 ± 42.1	11.1	66
	4	10	10/10	29.2 ± 8.9	110.1 ± 33.1	10.0	59
SL-Push (sim)	3	10	8/10	30.0 ± 46.3	193.3 ± 111.3	31.1	62
SL-Push (sim)	4	10	8/10	23.4 ± 18.2	171.7 ± 61.2	20.5	41
DFS-WCCG	3	10	10/10	142.6 ± 183.1	178.1 ± 72.1	13.9	762
DFS-WCCG	4	10	9/10	101.9 ± 103.9	175.9 ± 113.6	14.3	498

Failure composition: SL-Push (sim) $N=3$ one plan failure and one timeout, $N=4$ two timeouts; DFS-WCCG $N=4$ one timeout; PushAround none.

Pre-fix history (superseded; baselines $n=5$): at $N=3$ PushAround 10/10, SL-Push (sim) 4/5, DFS-WCCG 3/5; at $N=4$ PushAround 7/10, SL-Push (sim) 0/5 (all plan failures), DFS-WCCG 4/5.

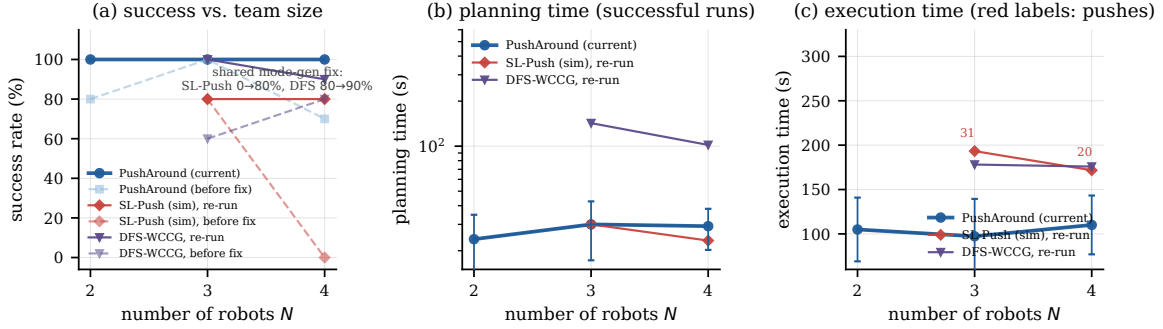


Figure 4: Robot-team scaling after the shared mode-generation refinement, with the pre-fix runs shown as dashed history. At $N=4$, the observed success rate is 100% for PushAround, 80% for SL-Push (sim), and 90% for DFS-WCCG. (b) Planning time and (c) execution time of successful runs; the annotations in (c) are the push counts of SL-Push (sim).

is descriptive: PushAround is the only method with 30/30 successes over $N=2, 3, 4$; at $N=3, 4$ it uses 29–30 s of planning and 97–110 s of execution, compared with 102–143 s and 176–178 s for DFS-WCCG, while SL-Push (sim) requires 172–193 s of execution and 20–31 pushes.

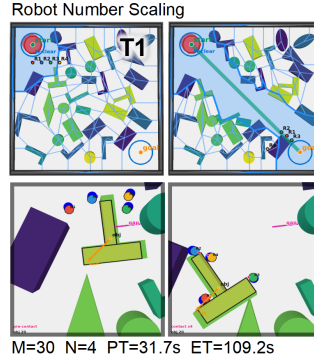


Figure 5: The four-robot study of the same evaluation ($M=30$, $N=4$): global initial and final states above, and two magnified views of a single pushing task (before contact and during the push) below.

Changes in the revised manuscript. Sec. III-C2 (Page 4) now describes the shared mode-generation

refinements and their preference for full-team assignments. Sec. IV-A6 and Table III (Page 7) report the post-fix $N \in \{2, 3, 4\}$ comparison for all policies, while Sec. III-E2 (Page 5) explains the bounded contact-assignment search:

- The revised text explains that bounded backtracking reduces the robot-order sensitivity of greedy replacement, and that a temporary per-task robot release is penalized so full-team modes remain preferred.

A bounded backtracking variant removes the robot-order sensitivity of this replacement. If no full-team assignment exists for a task, one robot is released for that single push and the full team is available again afterwards.

- The robot-team group of Table III reports success, planning time, and execution time for PushAround and both baselines at $N=3$ and $N=4$, using 10 seeds per cell.
- Sec. III-E2 contrasts the $N|\mathcal{A}|$ greedy stage with the $|\mathcal{A}|^N$ exhaustive assignment space and notes the finite backtracking budget.

the greedy stage evaluates at most $N|\mathcal{A}|$ single-contact substitutions instead of the exhaustive $|\mathcal{A}|^N$ assignment, and when it fails, a bounded-backtracking search expands at most $B=2000$ partial assignments, a fixed budget independent of the team size. The per-task cost is therefore at most $(N|\mathcal{A}|+2B)$ LP feasibility checks, rather than growing as $|\mathcal{A}|^N$.

Table 6: Complete robot-team-scaling grid ($M=30$, nominal physics), all cells on the post-fix code with 10 seeds. The pre-fix rows are kept only as history.

Policy	N	Trials	Succ.	PT (s)	ET (s)	pushes	sims
PushAround (current)	2	10	10/10	23.9 ± 10.9	105.0 ± 35.9	11.7	81
PushAround (current)	3	10	10/10	30.0 ± 12.7	97.3 ± 42.1	11.1	66
PushAround (current)	4	10	10/10	29.2 ± 8.9	110.1 ± 33.1	10.0	59
SL-Push (sim)	3	10	8/10	30.0 ± 46.3	193.3 ± 111.3	31.1	62
SL-Push (sim)	4	10	8/10	23.4 ± 18.2	171.7 ± 61.2	20.5	41
DFS-WCCG	3	10	10/10	142.6 ± 183.1	178.1 ± 72.1	13.9	762
DFS-WCCG	4	10	9/10	101.9 ± 103.9	175.9 ± 113.6	14.3	498
<i>Pre-fix history (superseded; baselines $n=5$)</i>							
PushAround	2	10	8/10	19.5	125.6	—	—
PushAround	3	10	10/10	21.8	100.6	—	—
PushAround	4	10	7/10	23.3	121.3	—	—
SL-Push (sim)	3	5	4/5	61.7	196.7	—	—
SL-Push (sim)	4	5	0/5	—	—	—	—
DFS-WCCG	3	5	3/5	164.6	199.8	—	—
DFS-WCCG	4	5	4/5	131.0	190.9	—	—

Q: What guarantees, if any, does the hybrid search provide regarding solution quality?

A: We appreciate this question. No global optimality or completeness guarantee is claimed, and the revision now states this explicitly in the formulation, node-selection, and termination discussions.

PIHS samples a finite set of pushing directions and contact modes, uses bounded gap lookahead, ranks nodes with a heuristic best-first priority whose remaining-cost term is *not* required to be admissible, and terminates at the first physics-validated W -clear solution. The returned schedule is therefore a feasible schedule under the evaluation model, not a globally minimal one. Eq. (5) is consequently presented as the clearance cost that ranks candidate schedules; it does not turn the implemented search into a solver of the underlying continuous hybrid optimization problem.

Completeness is also not guaranteed: a feasible scene may require an unsampled pushing direction, contact assignment, or deeper recursion, or may exceed the finite computation budget. The node-expansion counts and ablations characterize empirical search efficiency and solution-finding reliability; they are not guarantees of solution quality.

Changes in the revised manuscript. Sec. II-D (Page 2), the node-selection discussion (Page 5), and the termination discussion (Page 5) now describe Eq. (5) as a ranking cost and explicitly state the limitations introduced by finite sampling, bounded lookahead, a non-admissible heuristic, and first-feasible termination:

- The formulation now describes Eq. (5) as ranking feasible schedules and limits the claim to simulator-validated feasibility.

Equation (5) ranks feasible schedules; PIHS uses finite sampling and does not guarantee global optimality of the continuous hybrid objective.

- The search and termination text state that finite sampling and a finite computation budget do not guarantee completeness.

The search operates on a finite sampled candidate set and therefore does not guarantee global optimality or completeness of the underlying continuous hybrid planning problem. (Sec. III-D) and the schedule π stored in ν is a feasible solution satisfying the terminal clearance condition and the validated pushing transitions, without implying that $\mathcal{J}_{\text{clr}}(\pi)$ is globally minimal. The objective covers the clearing phase only, since the external vehicle traverses the corridor after clearing completes. (termination)

- The node-selection text identifies the remaining-cost term as a non-admissible heuristic and makes clear that the priority does not provide an A* optimality guarantee.

The remaining-cost term is a heuristic and is not required to be admissible. The score prioritizes nodes by realized cost and promising future actions, but it does not provide an A* optimality guarantee.

Q: The objective includes the total duration T and the control-effort cost J . Is the time needed for the final vehicle to actually traverse the cleared corridor included in this objective, or is the objective only concerned with clearing the obstacles? Similarly, does J only capture the robots' effort in moving the obstacles?

A: Yes. The objective covers the robotic clearance operation only, while the vehicle traversal appears only through the terminal clearance condition.

- $T \triangleq \sum_{k=1}^K \Delta t_k$ is the duration of robotic clearance only; the external vehicle is not moving concurrently and traverses the corridor after clearing is complete.
- $J(\cdot)$ is the simulation-estimated manipulation effort for moving the obstacles. Vehicle traversal time is not part of the objective and is represented only by the terminal W -clearance condition.

Changes in the revised manuscript: Sec. II-D (Page 2) now defines T as the robotic clearance duration, defines $J(\cdot)$ as the simulation-estimated manipulation effort, and states that the external vehicle traverses only after clearing and is represented through the terminal W -clearance condition.

Q: *The paper demonstrates online replanning when the real execution differs from the simulation, How can simple pushing robots evaluate this discrepancy?*

A: We agree that the previous wording (“the realized motion deviates from simulation”) was too vague and could be read as requiring online parameter identification or force sensing. That is not the implementation, and the revision replaces the phrase with an explicit state-based monitor.

In the hardware experiments, motion capture provides robot and object poses. A push is declared to deviate — and replanning is triggered from the observed state — when the measured object pose departs from the successor state predicted by EvalSim by more than **0.3 m in position** or $\pi/4$ **in orientation**, or when the transition/contact checks fail (blocked pre-contact transition, contact loss or stall, unsatisfied geometric task predicate, invalidated WCCG connectivity for the remaining schedule). The pose threshold is deliberately loose: a slightly larger or slightly off-axis push is accepted rather than aborted, because such deviations are routinely recoverable by the subsequent tasks. No onboard force sensing or mass/friction estimation is required.

Changes in the revised manuscript: Sec. IV-B1 (Page 7) now defines the pose-based deviation thresholds used by the motion-capture monitor, while Sec. III-E1 lists the additional contact, stall, task-predicate, and connectivity triggers.

Q: *Overall, this is a good contribution to the field and a well-written paper. The combination of geometric reasoning, recursive pushing, and physics-informed search is interesting, and the real-world validation strengthens the contribution.*

A: We thank the reviewer for the encouraging assessment. The revision leaves the planning framework, the recursive-pushing mechanism and the hardware validation unchanged, and concentrates on the clarifications and the additional evaluations requested in the review. Every number quoted above is reproduced from the per-trial records of the revised runs: the evaluation package accompanying this response contains the raw logs, the resolved configuration files and the complete per-condition tables, and Sec. IV-A1 of the manuscript states the shared protocol (timeout, control-step budget and reachability checks) that applies to PushAround and to both baselines alike. The recursive-pushing mechanism that the reviewer highlights is also the main reason the dense-clutter runs recover once a direct push is blocked; the no-recursion ablation in Sec. IV-A6 quantifies how much of the reported success rate depends on it.

To Reviewer 4

Q: *This paper presents PushAround, a physics-informed hybrid search method for collaborative path clearing. The method uses a W-clearance connectivity graph to identify blocking gaps, ranks these gaps, and generates multi-robot pushing actions that are checked using short-horizon physics simulation. The paper includes simulation comparisons, ablation studies, scalability tests, and hardware experiments.*

Overall, I think the paper addresses an interesting problem and the proposed framework is practical. The combination of geometric reasoning and physics simulation is useful, and the experimental results are encouraging. I would support acceptance after revision, but I think a few parts of the method need some clarification.

A: We thank the reviewer for the supportive assessment and for pinpointing four places where the presentation was under-specified. Each is addressed below with the corresponding revised text. The supplementary evaluation package has also been updated where relevant.

Q: *First, the WCCG connectivity condition is important since it is used throughout the search and also determines termination. The intuition is reasonably clear, but I think the paper would benefit from a little more justification here, or perhaps a comparison with a standard configuration-space connectivity check. This seems especially relevant when the obstacles are non-convex.*

A: We agree, and we provide both the requested comparison and an empirical validation. Standard alternatives first inflate the obstacles by $W/2$ and then test connectivity on an occupancy grid (BFS/A* or a distance transform) or on a sampling-based roadmap (PRM/RRT-style). These approaches approximate the answer at a chosen grid resolution or sample density and generally return a binary connectivity result; the representation must also be updated as obstacles move. WCCG instead builds an embedded graph directly from the clearance-relevant geometry. Its annotated bridges expose the separating bottleneck and frontier sequence that PIHS uses to choose the next obstacle, in addition to answering the connectivity query. The same-face test is the graph query used to evaluate the terminal condition; its validity follows from the $W/2$ inflation interpretation under the stated decomposition. Non-convex obstacles are handled by the clearance-relevant convex decomposition described in Sec. III-A2 (Page 3).

Because geometric degeneracies and decomposition choices still matter in practice, we compare WCCG against two independent references on 300 random scenes stratified into convex, mixed, and non-convex obstacles: a continuous configuration-space (CC) reference and a rasterized reference (FR) at 0.01 m oracle resolution, with a coarser 0.02 m raster used only as a stability check. CC and FR disagree in **4/300** cases, all of which are raster-unstable near narrow or tangential contacts. On all 300 scenes, WCCG agrees with the continuous reference and produces **zero false positives and zero false negatives**; a false positive would be the dangerous case because it could terminate PIHS prematurely. WCCG answers a query in **1.07 ms** on average, compared with 1.36 ms for CC and 243.1 ms for FR. The latter ratio is an implementation-level **227 $\times$** speed difference, and WCCG additionally supplies the gap graph consumed by PIHS.

Changes in the revised manuscript. Sec. III-A and Sec. III-A2 (Page 3) now identify occupancy-grid and sampling-based connectivity checks as the main alternatives and explain why WCCG exposes the bottleneck structure needed by PIHS. Sec. IV-A3 and Table I (Page 6) add the 300-scene comparison and the timing scope:

- The manuscript now derives the criterion from configuration-space inflation, describes the clearance-relevant treatment of non-convex obstacles, and states that consistency with CC and

Table 7: WCCG connectivity validation on 300 random scenes (10 movable obstacles, 8×8 m, clearance $W=1$ m). FP/FN are WCCG errors with respect to the continuous reference. Times are means over 100 scenes per category.

Category	CC=FR	WCCG vs. CC			Time (ms)			FR/WCCG
		agree	FP	FN	CC	FR	WCCG	
Convex	100/100	100/100	0	0	1.23 ± 0.13	282.9 ± 52.3	0.86 ± 0.15	330
Mixed	98/100	100/100	0	0	1.33 ± 0.13	242.4 ± 55.7	1.05 ± 0.17	231
Non-convex	98/100	100/100	0	0	1.51 ± 0.18	204.1 ± 51.4	1.30 ± 0.19	156
Total	296/300	300/300	0	0	1.36 ± 0.19	243.1 ± 62.0	1.07 ± 0.25	227

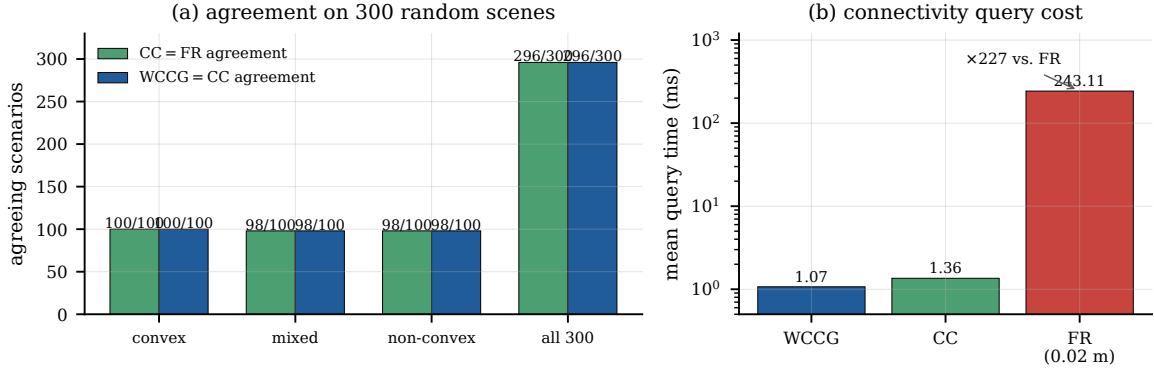


Figure 6: WCCG validation. (a) Agreement per category: CC and FR differ on 4 of 300 scenes (all raster-unstable), whereas WCCG matches the continuous reference on all 300 with zero false positives/negatives. (b) Mean connectivity-query cost on a logarithmic scale.

FR is evaluated numerically.

The criterion follows the standard configuration-space inflation interpretation: expanding obstacles by a disk of radius $W/2$ turns gaps narrower than W into overlapping barriers. Non-convex obstacles are handled through clearance-relevant decomposition, after which WCCG records the separating bottlenecks.

- Table I reports agreement, false-positive/false-negative counts, and runtime on the same 300 scenes; its note specifies that the timing comparison uses one harness and a compiled WCCG backend.
- The text now says that WCCG avoids introducing a workspace-grid resolution into repeated queries, while still depending on a correct decomposition and bridge construction.

Conventional clearance checks either rasterize the workspace into an inflated occupancy grid or query sampling-based roadmaps, and must therefore fix a resolution and re-query the map whenever an obstacle moves. Built directly from obstacle geometry, the WCCG introduces no grid-resolution parameter in these repeated queries.

Q: Second, I was a little confused about the relationship between Eq. (5) and the actual search procedure. Eq. (5) is written as an optimization problem, while the implementation uses sampled pushing directions, limited lookahead, and a heuristic best-first search that terminates once a feasible solution is found. The authors should clarify whether any optimality or completeness claim is intended. If not, it may be better to describe Eq. (5) as the objective used to guide the search.

A: We agree with the reviewer’s suggested framing. Eq. (5) is now introduced as the clearance cost $\mathcal{J}_{\text{clr}}(\pi)$ that ranks candidate schedules. PIHS is a feasibility-oriented heuristic best-first search over a finite sampled set of pushing directions and contact modes; it uses bounded lookahead and stops at the first simulator-validated W -clear solution. The remaining-cost term is not required to be admissible, so the first feasible schedule is not claimed to minimize $\mathcal{J}_{\text{clr}}$.

Completeness is likewise not guaranteed. A feasible scene may require an unsampled direction, contact assignment, or deeper recursion, or may exceed the finite computation budget. The returned schedule is guaranteed only to satisfy the terminal clearance condition and the validated transitions under the evaluation model. The empirical node-expansion counts and ablations describe search efficiency and solution-finding reliability; they do not constitute a formal solution-quality guarantee. These distinctions are now stated in Secs. II-D and III-D (Pages 2 and 5).

Q: *Third, it is not completely clear how the optimized contact forces are used on the hardware. The pushing mode includes both contact points and forces, but the execution section mainly describes moving the robots to the contact configuration and then following the desired pushing direction. It would be useful to briefly explain the low-level controller and whether these forces are actually commanded or are mainly used to check feasibility. Related to this, I am also not sure whether coupled motion between multiple objects, such as in the second hardware experiment, is explicitly considered during planning or is mostly handled by the physics simulation.*

A: Thank you for raising both points. The optimized forces are not commanded on the hardware; they are used to certify wrench feasibility, while the hardware tracks a short-horizon object-motion reference generated by the simulator.

For a candidate contact tuple, the LP checks whether the robots can generate a wrench compatible with the desired object motion within the modeled force, friction, and unilateral-contact constraints. The optimized force vector is therefore a feasibility and ranking variable, not a hardware setpoint. Over the next control interval, EvalSim predicts a short forward reference (a few centimetres and a few degrees); the velocity-level controller converts this reference and the motion-capture pose error into planar velocity commands for the Mecanum UGVs. Feedback and replanning absorb the difference between the reference and the realized motion.

Coupled motion is handled by the rollout. Although a high-level strategy names one directly targeted obstacle, EvalSim advances all robots and all movable bodies jointly; object–object contacts are resolved by the physics engine and the complete successor scene is returned to the search. The adjacent T-shaped block in hardware Scenario 2 therefore moves as a contact-induced outcome rather than as a separately scripted task.

Changes in the revised manuscript. The manuscript now clarifies, in Secs. II-C, III-C3, III-E1, and IV-B1–B2 (Pages 2, 4–7), that the optimized force is a feasibility variable, the controller tracks a short-horizon simulator reference using pose feedback, and coupled object motion is resolved by the joint rollout:

- The force-feasibility, velocity-control, motion-capture, and joint-rollout interfaces are now described explicitly in the corresponding sections.

The wrench feasibility encoded in ξ acts as a screening condition rather than a force setpoint: it certifies that the chosen contact configuration can realize the required object motion within the robots’ actuation limits. (Sec. III-C) and a mode is retained only if its object wrench lies within the robots’ actuation limits, and the controller tracks the planned contact poses and pushing velocities with feedback. Planning therefore runs in simulation through EvalSim, and every executed push returns an observed state that re-enters the planner. (Sec. III-E1)

- The hardware discussion now states that the neighboring-object motion in Scenario 2 is an emergent outcome of EvalSim and is fed back through the updated scene and WCCG.

The two Mecanum UGVs are velocity-controlled, and the optimized contact forces are used only for feasibility evaluation rather than direct motor control. A push triggers replanning only when the measured object position error exceeds 0.3 m or its orientation error exceeds $\pi/4$. (Sec. IV-B1) and This neighboring-object motion is not a separately encoded action but an emergent outcome of the coupled rigid-body rollout in EvalSim, whose snapshot is used to update the WCCG and continue replanning. (Sec. IV-B2)

Q: *The experiments are generally convincing. The paper already mentions that the baselines use the same W-clearance criterion, object distribution, and execution simulator. However, a few more implementation details, such as robot reachability checks, timeout limits, and computation budget, would make the comparison easier to interpret. I would also suggest adding error bars or standard deviations, particularly for the scalability experiment where only five seeds are used for each obstacle density.*

A: We have made the protocol explicit and now report dispersion throughout. The full protocol is summarized in Table 8.

- A contact candidate is retained only if the associated pre-contact pose is collision-free and the transition planner can reach it from the current robot configuration; a multi-robot tuple is rejected if any assigned robot fails this check. All compared methods use this same check, together with the same footprints, obstacle realizations, success criterion, and execution simulator.
- All suites use a per-trial timeout of **800 s**, a per-trial cap of **20,000** simulator control steps, and **8** parallel workers; plan failures, timeouts and timeout-pruned trials are assigned the 800 s value before averaging planning times, and the failure composition is reported per condition.
- The density sweep, the robot-team study (all policies) and the mismatch study for PushAround use 10 seeds per cell; only the *mismatch* baseline supplements use 5 seeds and are labelled as such. Fig. 7 reports Wilson 95% intervals for every headline number, and Table III in the manuscript reports $\text{mean} \pm \text{s.d.}$
- The original manuscript stated a 400 s cap. The stored logs and the revised protocol use 800 s; the earlier statement and the corresponding failure-time imputation have been corrected.

Changes in the revised manuscript: Sec. IV-A1 (Page 6) now specifies the common footprints, scene configurations, collision and reachability checks, simulator, 800 s timeout, and 20,000-step simulator cap. It also states how unsuccessful trials enter the capped planning-time statistics.

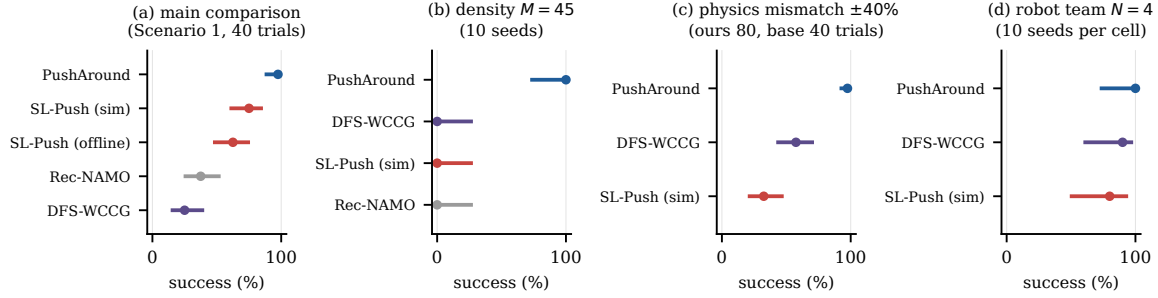


Figure 7: Wilson 95% confidence intervals of the headline success rates. The intervals make sampling uncertainty explicit: the density and mismatch comparisons show clear separation, whereas some robot-team cells overlap and are interpreted descriptively.

Table 8: Protocol and budget summary for all reported studies. All suites share the simulator, the success criterion, and the collision/reachability checks.

Study	Workspace	M	N	Seeds	Timeout / cap	Workers
Main comparison (Table II)	8×5 m	10	2	40 trials	800 s / 20k	8
Density (Table III, Fig. 10)	8×8 m	15–45	2	10 per density	800 s / 20k	8
Mismatch (Table III)	8×8 m	30	2	10 (ours) / 5 (baseline)	800 s / 20k	8
Robot scaling (Table III)	8×8 m	30	2–4	10 per cell	800 s / 20k	8
WCCG validation (Table I)	8×8 m	10	—	300 scenes	n/a	4 (C++)

Scene generation in the generalization studies uses movable-object masses sampled from $[20, 30]$ kg; the Scenario-1 comparison uses $[5, 15]$ kg. PushAround’s density results are produced with a precomputed mode table (`use_mode_table`), whereas the mismatch and robot-scaling studies use greedy mode optimization; this explains the small planning-time differences between the corresponding nominal rows (e.g. 11.7 s versus 16.8 s at $M=30$, $N=2$) and does not affect success rates.

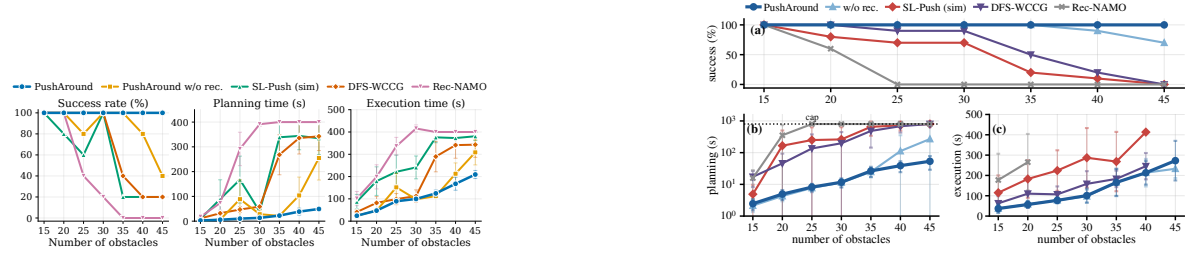
Q: Overall, I think the approach is technically reasonable and the experiments support the proposed method. My comments are mainly about clarification rather than major technical concerns, and I would support acceptance after these revisions.

A: We thank the reviewer for the supportive assessment. The four clarifications above are addressed without changing the method, and the added protocol details, dispersion statistics and the configuration-space comparison are intended to make the evaluation easier to interpret. The figures that changed in this revision are reproduced side by side with their original versions in the following section, so that all four changes can be checked at a glance, and the complete data tables for each study are placed next to the reply they support rather than in an appendix.

Updated and added figures

Three figures of the manuscript changed in this revision. In each pair the left image is the original version (first submission) and the right image is the revised version; the page refers to the location in the revised manuscript.

F1. Obstacle-density scaling: revised Fig. 10 (Page 7)



(a) Original: five seeds per density, mean only, 400 s cap.

(b) Revised: ten seeds per density, mean $\pm$ s.d., 800 s cap.

Figure 8: Obstacle-density scaling, original (left) versus revised (right). The panel structure is unchanged, so the two images are directly comparable: the revision doubles the seeds per density, adds dispersion to the timing panels, and uses the actual 800 s timeout instead of the 400 s cap of the summary.

F2. Simulation snapshots: revised Fig. 7 (Page 6)

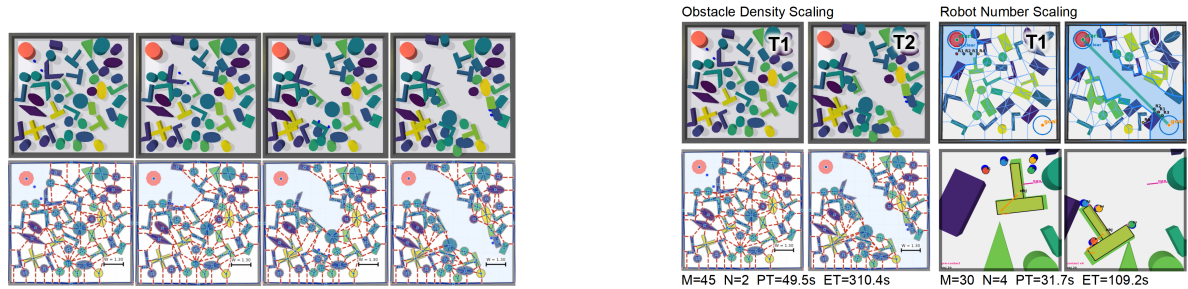


(a) Original: one overlay inset did not match its snapshot.

(b) Revised: regenerated with the correct inset.

Figure 9: Execution and planning alignment in the two simulation scenarios, original (left) versus revised (right). One inset of the original composite was incorrect: the fourth sub-image of its bottom row duplicated the sixth, so that WCCG overlay did not match the snapshot above it. The panel has been regenerated; the caption and the message are unchanged.

F3. Scalability scenes: revised Fig. 9 (Page 7)



(a) Original: obstacle-density run only.

(b) Revised: density run (left half) plus the four-robot study (right half).

Figure 10: Simulation scenes of the scalability evaluation, original (left) versus revised (right). The revised figure covers both axes instead of density alone: the left half keeps the representative $M=45$, two-robot run (snapshots above, WCCG overlays below), and the right half adds the four-robot study ($M=30$, $N=4$) with initial and final states above and two magnified views of one pushing task below, illustrating the team-size results of Table III (Page 7).